

Rick Sarkar

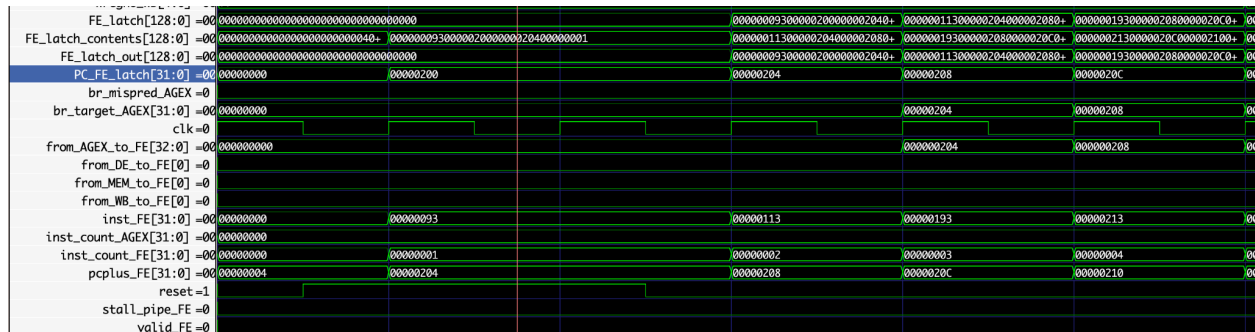
Professor Lin

9 February 2026

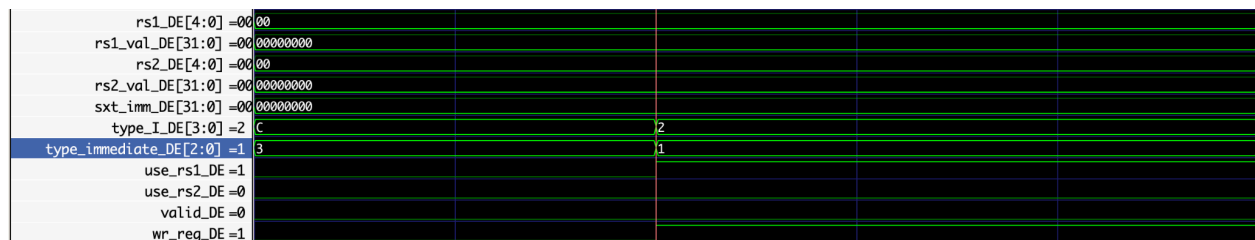
CS 3220 Lab 1 Writeup

Q1: Explain the actions in each pipeline stage while executing test/part1/test1.mem.

FE: In the FE stage, we fetch the instruction using our program counter, which points to where the instruction is stored in memory. We can observe the waveform where PC_FE_latch increments by 4, since each instruction is 4 bytes, beginning at x00000200. We can also observe the inst_FE signal which is x93, which is the value of the fetched instruction. This will be decoded in the next stage of our pipeline.



DE: In this stage, we take the binary value of our instruction and interpret it as an instruction according to the specified Instruction Set Architecture. In this case, we are dealing with an addi instruction, which adds a register value with an immediate, and places the result into another register. We can verify inst_DE and PC_DE have the correct values, equal to those from FE's waveform. The DE stage is where the registers that serve as operands are selected appropriately.

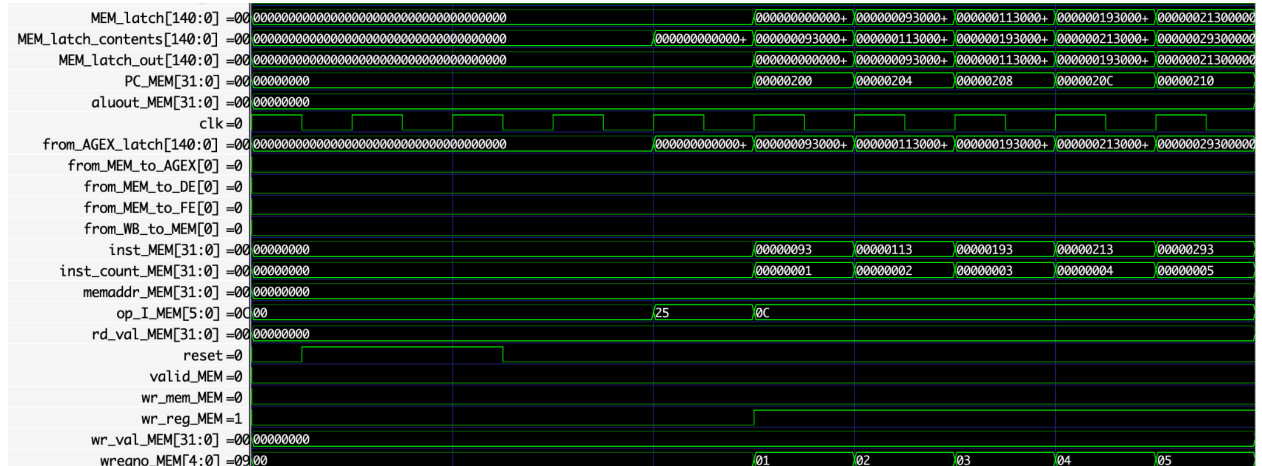


DE_latch[237:0] =00	00000072600000438000004386+	00000082600000438000004406+	00000092600000440000004486+	000000A2600000448000004506+	000000B260000045000000458600
DE_latch_contents[237:0] =00	00000082600000438000004406+	00000092600000440000004486+	000000A2600000448000004506+	000000B2600000450000004586+	000000C260000045800000460600
DE_latch_out[237:0] =00	00000072600000438000004386+	00000082600000438000004406+	00000092600000440000004486+	000000A2600000448000004506+	000000B260000045000000458600
F3_DE[2:0] =00	000				
F7_DE[6:0] =00	00				
PC_DE[31:0] =00	0000021C	00000220	00000224	00000228	0000022C
br_mispred_AGEX =0					
clk =1					
from_AGEX_to_DE[0] =0					
from_DE_to_FE[0] =0					
from_FE_latch[128:0] =00	0000004130000021C000002200+	00000049300000220000002240+	00000051300000224000002280+	000000593000002280000022C0+	0000006130000022C00000230000
from_MEM_to_DE[0] =0					
from_WB_to_DE[37:0] =00	2500000000	2600000000	2700000000	2800000000	2900000000
has_data_hazards =0					
in_use_regs[31:0] =00	000000E0	000001C0	00000380	00000700	00000E00
inst_DE[31:0] =00	00000413	00000493	00000513	00000593	00000613
inst_count_DE[31:0] =00	00000008	00000009	0000000A	0000000B	0000000C
is_br_DE =0					
op_DE[6:0] =13	13				
op_I_DE[5:0] =0C	0C				
pcplus_DE[31:0] =00	00000220	00000224	00000228	0000022C	00000230
pipeline_stall_DE =0					
rd_DE[4:0] =01	08	09	0A	0B	0C

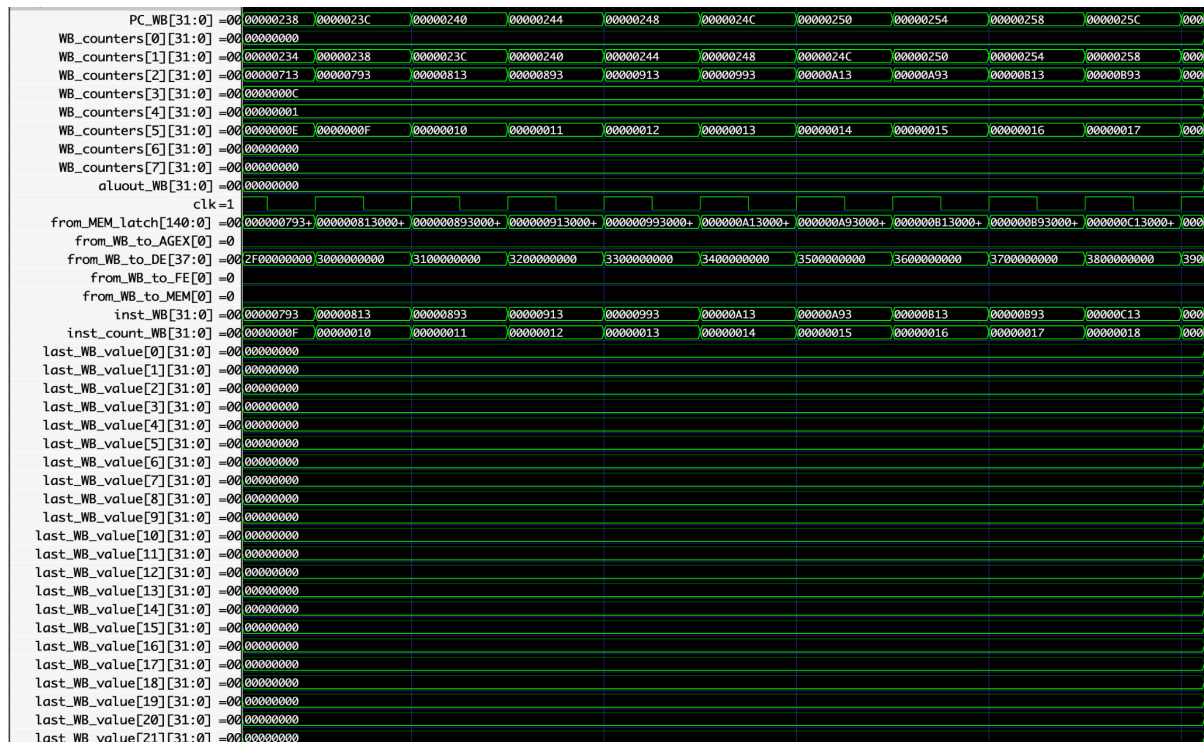
AGEX: The AGEX stage contains the bulk of the logic done. Arithmetic operations, including addi, are computed and the result is saved in aluout_AGEX to be later written to the destination registers. The Arithmetic Logic Unit can also be used for address computations to load values from or branch/jump. We can verify the lack of branch behavior from the signals in the waveform below.

Time	56 ps	64 ps	72 ps	80 ps	88 ps
AGEX_latch_out[140:0] =00	000000+ 0000008513000+ 000000593000+ 000000613000+ 000000693000+ 000000713000+ 000000793000+ 000000813000+ 000000893000+ 000000913000+ 000000930000				
FE_latch_out[128:0] =00	000000+ 000000613000+ 000000693000+ 000000713000+ 000000793000+ 000000813000+ 000000893000+ 000000913000+ 000000930000+ 000000A13000+ 000000A30000				
MEM_latch_out[140:0] =00	000000+ 000000493000+ 000000513000+ 000000593000+ 000000613000+ 000000693000+ 000000713000+ 000000793000+ 000000813000+ 000000893000+ 000000913000+ 000000930000				
clk =1					
cycle_count[31:0] =00	000000+ 0000000C 0000000D 0000000E 0000000F 00000010 00000011 00000012 00000013 00000014 00000015				
from_AGEX_to_DE[0] =0					
from_AGEX_to_FE[32:0] =00	000000+ 00000022C 000000230 000000234 000000238 00000023C 000000240 000000244 000000248 00000024C 000000250				
from_DE_to_FE[0] =0					
from_MEM_to_AGEX[0] =0					
from_MEM_to_DE[0] =0					
from_MEM_to_FE[0] =0					
from_WB_to_AGEX[0] =0					
from_WB_to_DE[37:0] =00	28000+ 29000000000 2A000000000 2B000000000 2C000000000 2D000000000 2E000000000 2F000000000 30000000000 31000000000 32000000000				
from_WB_to_FE[0] =0					
from_WB_to_MEM[0] =0					
AGEX_latch[140:0] =00	000000+ 0000008513000+ 000000593000+ 000000613000+ 000000693000+ 000000713000+ 000000793000+ 000000813000+ 000000893000+ 000000913000+ 000000930000				
AGEX_latch_contents[140:0] =00	000000+ 000000593000+ 000000613000+ 000000693000+ 000000713000+ 000000793000+ 000000813000+ 000000893000+ 000000913000+ 000000930000+ 000000A13000+ 000000A30000				
AGEX_latch_out[140:0] =00	000000+ 000000513000+ 000000593000+ 000000613000+ 000000693000+ 000000713000+ 000000793000+ 000000813000+ 000000893000+ 000000913000+ 000000930000				
PC_AGEX[31:0] =00	000000+ 00000228 0000022C 00000230 00000234 00000238 0000023C 00000240 00000244 00000248 00000250				
aluout_AGEX[31:0] =00	00000000				
br_cond_AGEX =0					
br_mispred_AGEX =0					
br_target_AGEX[31:0] =00	000000+ 0000022C 00000230 00000234 00000238 0000023C 00000240 00000244 00000248 0000024C 00000250				
clk =1					
from_AGEX_to_DE[0] =0					
from_AGEX_to_FE[32:0] =00	000000+ 00000022C 000000230 000000234 000000238 00000023C 000000240 000000244 000000248 00000024C 000000250				
from_DE_latch[237:0] =00	000000+ 000000826000+ 0000008C26000+ 0000009D26000+ 000000AE26000+ 000000BF26000+ 000001026000+ 000001126000+ 000001226000+ 000001326000+ 000001426000				
from_MEM_to_AGEX[0] =0					
from_WB_to_AGEX[0] =0					
inst_AGEX[31:0] =00	000000+ 00000593 00000613 00000693 00000713 00000793 00000813 00000893 00000913 00000993 00000A13				
inst_count_AGEX[31:0] =00	000000+ 00000008 0000000C 0000000D 0000000E 0000000F 00000010 00000011 00000012 00000013 00000014				
is_br_AGEX =0					
op_I_AGEX[5:0] =00	0C				
pcplus_AGEX[31:0] =00	000000+ 0000022C 00000230 00000234 00000238 0000023C 00000240 00000244 00000248 0000024C 00000250				
regval1_AGEX[31:0] =00	00000000				
regval2_AGEX[31:0] =00	00000000				
reset =1					
sxt_imm_AGEX[31:0] =00	00000000				
valid_AGEX =0					
wr_reg_AGEX =0					

MEM: The MEM stage is used when we are writing or reading from memory. For test 1, it is not relevant since we are performing operations between registers and storing the final result in another register. At no point do we read from or write to a memory address, meaning the MEM stage is not utilized. This can be verified from the lack of signals set in the MEM stage, save for `wr_reg_MEM` since we plan to overwrite a register value in the next stage of our pipeline.



WB: The WB, or write back, stage involves writing the results from the AGEX and MEM stages to the actual registers that need updating. In the waveform, we can observe the ALU result calculated in the AGEX stage being written back to the GP register.



Read-After-Write Hazards occur when an instruction that updates a register is immediately followed by another that uses its value as an operand. Since the WB (writeback) stage has not been completed by the time the register value is read, we must stall the pipeline in order to prevent an old value from being used over the updated one. The RISC-V processor utilizes **data forwarding**, which we can verify using from DE_latch_contents/DE_latch_out. Data forwarding involves sending computed values forward for the next instruction to use, rather than reading from a deprecated register that WB hasn't reached yet.

[illegible]

Q3: Explain how your RISC-V processor handles branch misprediction in test/part1/test4.mem.

Since this lab's processor defaults to assuming that branches are not taken, branch mispredictions occur every time a branch is indeed taken. This means that our FE stage will always fetch the next sequential instruction (PC + 4) since we never attempt to predict that the next instruction lies at the branch's destination. Later, however, our AGEX stage calculates the branch condition and determines that it will be taken. Then, signals such as `br_taken` become set, and our branch target value is set to the target PC based on the type of branch instruction involved.

PC_FE_latch[31:0]	0000024C		00000250		
br_mispred_AGEX					
br_target_AGEX[31:0]	00000248		0000024C		
clk					
from_AGEX_to_FE[32:0]	000000248		00000024C		
from_DE_to_FE[0]					
from_MEM_to_FE[0]					
from_WB_to_FE[0]					
inst_FE[31:0]	00000A13		00000A93		
inst_count_AGEX[31:0]	00000000				
inst_count_FE[31:0]	00000014		00000015		
pcplus_FE[31:0]	00000250		00000254		
reset					
stall_pipe_FE					
valid_FE					
MEM_latch[140:0]	0000008130000023C3000000040000000030		000000893000002403000000040000000031		
MEM_latch_contents[140:0]	000000893000002403000000040000000031		000000913000002443000000040000000032		
MEM_latch_out[140:0]	0000008130000023C3000000040000000030		000000893000002403000000040000000031		
PC_MEM[31:0]	00000240		00000244		
aluout_MEM[31:0]	00000000				
clk					
from_AGEX_latch[140:0]	000000893000002403000000040000000031		000000913000002443000000040000000032		
from_MEM_to_AGEX[0]					
from_MEM_to_DE[0]					
from_MEM_to_FE[0]					
from_WB_to_MEM[0]					
inst_MEM[31:0]	00000893		00000913		
inst_count_MEM[31:0]	00000011		00000012		
memaddr_MEM[31:0]	00000000				
op_I_MEM[5:0]	0C				
rd_val_MEM[31:0]	00000000				
reset					
valid_MEM					
wr_mem_MEM					
wr_reg_MEM					
wr_val_MEM[31:0]	00000000				
wregno_MEM[4:0]	11		12		
PC_WB[31:0]	0000023C		00000240		
WB_counters[0][31:0]	00000000				
WB_counters[1][31:0]	00000238		0000023C		